

ChipMIT Working Committee — Projects 2026

---

# Computer Architecture

8-bit ALU with Manchester Carry Chain Adder

---

**Aarushi Gupta**

B.Tech. ECE

Reg. No.: 251090070190

# 1. Introduction

This project involves designing an 8-bit Arithmetic Logic Unit (ALU) using the Manchester Carry Chain mechanism for addition. The design is implemented in Verilog HDL and simulated using Icarus Verilog, with waveforms verified in GTKWave.

An ALU is the heart of any processor — it performs all arithmetic and logical computations. The choice of adder architecture directly impacts speed, and the Manchester Carry Chain eliminates the sequential carry delay found in a simple ripple carry adder.

# 2. What is a Manchester Carry Chain Adder?

## The Problem with Ripple Carry

In a standard ripple carry adder, each bit stage must wait for the carry output of the previous stage. For 8 bits this means carry propagates through 8 sequential gate delays — a delay that grows linearly with bit width.

## How Manchester Carry Works

The Manchester Carry Chain computes all carries in parallel using two signals per bit:

| Signal        | Formula                | Meaning                                                                |
|---------------|------------------------|------------------------------------------------------------------------|
| Generate (G)  | $G = A \text{ AND } B$ | This bit will always produce a carry, regardless of the incoming carry |
| Propagate (P) | $P = A \text{ XOR } B$ | This bit will pass along an incoming carry if one exists               |

Using G and P, every carry is expressed as a direct combinational formula:

```
C1 = G0 | (P0 & Cin)
C2 = G1 | (P1 & G0) | (P1 & P0 & Cin)
...
sum = P ^ C[7:0]
```

All 8 carry values are computed simultaneously — no stage waits for the previous one. This makes it significantly faster than ripple carry, especially for wider data buses.

# 3. ALU Operations

The ALU supports 8 operations selected via a 3-bit opcode. Two Manchester Carry Adder instances are used — one for ADD and one for SUB.

| Opcode | Op  | Description                        |
|--------|-----|------------------------------------|
| 000    | ADD | A + B using Manchester Carry Adder |

|            |            |                                              |
|------------|------------|----------------------------------------------|
| <b>001</b> | <b>SUB</b> | A - B via 2's complement: $A + (\sim B) + 1$ |
| <b>010</b> | <b>AND</b> | Bitwise A AND B                              |
| <b>011</b> | <b>OR</b>  | Bitwise A OR B                               |
| <b>100</b> | <b>XOR</b> | Bitwise A XOR B                              |
| <b>101</b> | <b>NOT</b> | Bitwise complement of A                      |
| <b>110</b> | <b>SHL</b> | Logical shift left by 1 ( $A \ll 1$ )        |
| <b>111</b> | <b>SHR</b> | Logical shift right by 1 ( $A \gg 1$ )       |

## Status Flags

Three flags are produced alongside every result:

| Flag          | Condition                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------|
| Zero Flag     | Set to 1 when the result is exactly 0                                                                  |
| Carry Out     | Set to 1 when there is an overflow or carry beyond 8 bits                                              |
| Negative Flag | Set to 1 when the MSB (bit 7) of the result is 1, indicating a negative value in signed representation |

## 4. Code Explanation

### manchester\_carry\_adder.v

This file has two modules. The first, `mcc_cell`, is a single-bit cell that outputs  $G = A \text{ AND } B$  and  $P = A \text{ XOR } B$ . The second is the full 8-bit adder: it instantiates eight `mcc_cell` units and computes all 8 carry signals simultaneously. The final sum is  $P \text{ XOR } C$  at each bit position.

```
assign c[1] = g[0] | (p[0] & c[0]);
assign c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & c[0]);
assign sum = p ^ c[7:0];
```

### alu\_8bit.v

The top-level module takes inputs A, B, and a 3-bit opcode. Two adder instances are created — one for addition ( $\text{cin}=0$ ) and one for subtraction ( $\sim B$ ,  $\text{cin}=1$ ). An `always` block selects the result based on the opcode.

```
case (op)
```

```

3'b000: begin result = add_result; carry_out = add_cout; end

3'b001: begin result = sub_result; carry_out = sub_cout; end

3'b010: begin result = a & b; end

...

endcase

```

### tb\_alu\_8bit.v (Testbench)

Drives the ALU through all 8 operations with test inputs, prints results to the terminal, and dumps all signal transitions to a .vcd file for GTKWave.

## 5. Simulation Results

### Terminal Output

All operations produce the expected results, confirming correctness of the design.

```

Command Prompt
=====
Compiling 8-bit ALU with Manchester Carry
=====
[OK] Compilation successful!
=====
Running Simulation...
=====
8-bit ALU Simulation - ChipMIT 2026
=====
VCD info: dumpfile alu_sim.vcd opened for output
ADD | A=45 + B=30 = 75 | Carry=0 | Zero=0
ADD | A=200 + B=100 = 44 | Carry=1 (overflow!)
SUB | A=50 - B=20 = 30 | Carry=1
SUB | A=25 - B=25 = 0 | Zero=1
AND | A=11001100 & B=10101010 = 10001000
OR  | A=11001100 | B=10101010 = 11101110
XOR | A=11001100 ^ B=10101010 = 01100110
NOT | ~A=11001100 = 00110011
SHL | A=5 << 1 = 10 | Carry(MSB)=0
SHR | A=20 >> 1 = 10 | Carry(LSB)=0
=====
Simulation Complete!
=====
tb_alu_8bit.v:85: $finish called at 1000000 (1ps)
=====
Opening Waveform in GTKWave...
=====
GTKWave Analyzer v3.3.100 (w)1999-2019 BSI
[0] start time.
[1000000] end time.

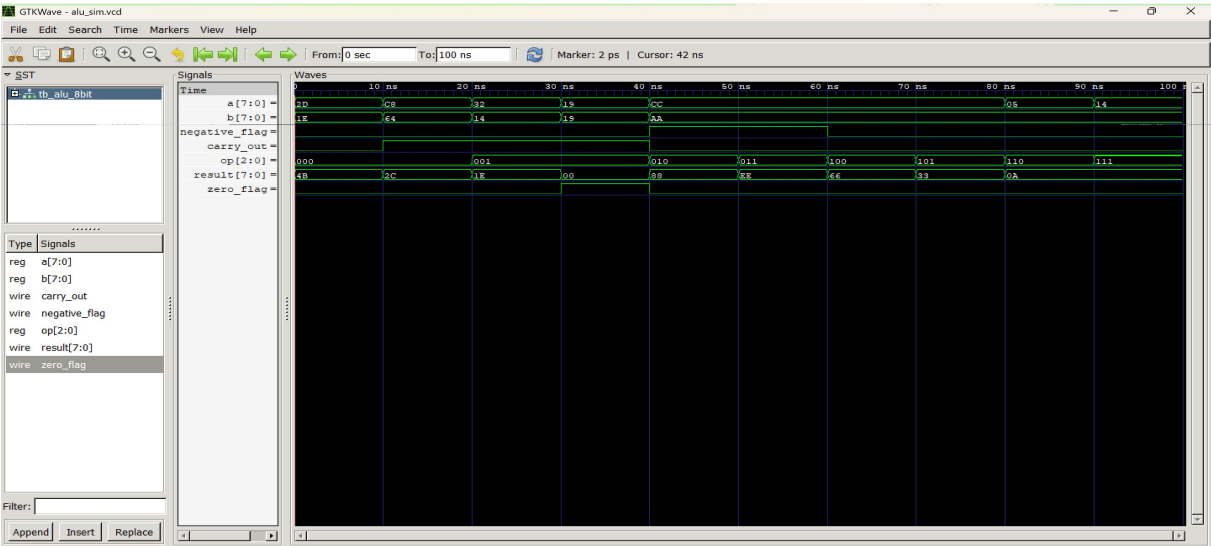
```

| Operation      | Input A | Input B | Result | Flags           |
|----------------|---------|---------|--------|-----------------|
| ADD            | 45      | 30      | 75     | Carry=0, Zero=0 |
| ADD (overflow) | 200     | 100     | 44     | Carry=1         |
| SUB            | 50      | 20      | 30     | Carry=1         |
| SUB (zero)     | 25      | 25      | 0      | Zero=1          |

|     |      |      |      |         |
|-----|------|------|------|---------|
| AND | 0xCC | 0xAA | 0x88 | —       |
| OR  | 0xCC | 0xAA | 0xEE | —       |
| XOR | 0xCC | 0xAA | 0x66 | —       |
| NOT | 0xCC | —    | 0x33 | —       |
| SHL | 5    | —    | 10   | Carry=0 |
| SHR | 20   | —    | 10   | Carry=0 |

GTKWave Waveform

The waveform shows op[2:0] stepping from 000 to 111 across the simulation timeline. The result updates at each transition, and carry\_out toggles correctly during overflow and subtraction.



6. Design Characteristics

| Parameter            | Value                                            |
|----------------------|--------------------------------------------------|
| Data Width           | 8 bits                                           |
| Adder Type           | Manchester Carry Chain (Carry Lookahead)         |
| Number of Operations | 8                                                |
| Opcode Width         | 3 bits                                           |
| Carry Computation    | Parallel — all 8 carries computed simultaneously |
| Subtraction Method   | 2's complement (A + ~B + 1)                      |

|                                |                           |
|--------------------------------|---------------------------|
| <b>Output Flags</b>            | Zero, Carry Out, Negative |
| <b>Implementation Language</b> | Verilog HDL               |
| <b>Simulation Tool</b>         | Icarus Verilog v12.0      |
| <b>Waveform Viewer</b>         | GTKWave v3.3.100          |

### Why Manchester Carry over Ripple Carry?

A ripple carry adder needs  $N$  sequential gate delays for an  $N$ -bit add — 8 delays here. The Manchester Carry Chain reduces this to a single combinational stage by expressing every carry as a direct boolean function of the original inputs. This is faster in hardware and is used in real processor designs.

## 7. Conclusion

---

The 8-bit ALU was successfully designed and verified using Verilog HDL. The Manchester Carry Chain adder computes all carry bits in parallel, making addition faster than a ripple carry design. All 8 operations — ADD, SUB, AND, OR, XOR, NOT, SHL, SHR — were tested and confirmed correct through both the Icarus Verilog terminal output and the GTKWave waveforms.

This project was a great introduction to hardware design thinking — how a low-level choice like carry architecture directly translates into faster, more efficient processors.